Blatt 08: Mini-Projekt C++-Interpreter

Carsten Gips, BC George (HSBI)

1 Aufgabe

Entwickeln Sie in Ihrem 3er-Team gemeinsam einen Interpreter für C++.

Nutzen Sie als Implementierungssprache Java.[1] Sie *können* ANTLR zur Erstellung Ihres Lexers und Parsers einsetzen, Sie können aber auch gern einen selbst implementierten LL-Parser mit *recursive descent* einsetzen.

Definieren Sie zunächst eine passende Grammatik und den AST, bevor Sie Lexer und Parser umsetzen. Achten Sie auf eine angemessene semantische Analyse, achten Sie dabei auf die in der Vorlesung besprochene C++-Semantik. Der Interpreter selbst soll ein einfacher Tree-walking Interpreter wie in der Vorlesung besprochen sein.

1.1 Initialisierung

Der Interpreter soll beim Start eine optional angegebene Datei mit C++-Code einlesen und verarbeiten können.

1.2 REPL

Danach soll dem User eine REPL angeboten werden, die einen Prompt in der Konsole ausgibt und in der weitere C++-Statements eingegeben werden können, die dann im Kontext des bisher verarbeiteten Codes interpretiert werden. Nach der Interpretation soll es jeweils eine Ausgabe des letzten Ergebnisses (oder Fehlers) auf der Konsole geben, und dann soll ein neuer Prompt ausgegeben und auf die nächste User-Eingabe gewartet werden.

Implementieren Sie eine spezielle Eingabe, mit der die REPL beendet werden kann.

Hinweis: Das Konzept einer speziellen Funktion ("`main()`") als Einstieg in die Ausführung des interpretierten C++-Programms funktioniert im Zusammenhang mit der REPL nicht wirklich. Interpretieren Sie die eingegebenen Statements entsprechend fortlaufend.

Hinweis: Achten Sie darauf, dass Ihr Interpreter die Eingabe nicht zu früh beendet und zu früh mit der Interpretation beginnt! Oft ist ein Zeilenumbruch das korrekte Signal zum Start der Verarbeitung. Es gibt aber auch Situationen (z.B. Funktionsdefinition o.ä.), wo ein Zeilenumbruch noch nicht eine vollständige Eingabe anzeigt. Hier soll nach einem Zeilenumbruch ein Hilfsprompt ausgegeben werden, damit der User weiss, dass die aktuelle Eingabe noch läuft.

1.3 Tests

Entwickeln Sie in Ergänzung zu den Vorgaben im Starter-Projekt weitere verschiedene Eingabebeispiele in unterschiedlicher Komplexität, mit denen Sie Ihren Interpreter testen können.

[1] Nach **vorheriger Absprache** können Sie auch eine andere Implementierungssprache verwenden.

1.4 Sprachumfang

Sie sollen mindestens folgende C++-Konzepte unterstützen:

- Basisdatentypen: `bool`, `int`, `char`
- Variablen
- Arrays
- C++-Referenzen
- Zuweisungen und Expressions
- Kontrollfluss: `if`-`then`-`else`, `while`-Schleifen
- Funktionen (Definition, Deklaration, Aufrufe)
- Klassen (mit Attributen und Methoden), Initialisierungslisten
- Einfach-Vererbung
- Polymorphie (dynamisch, statisch)
- Eingebaute Funktionen: `print_bool`, `print_int`, `print_char` (Ausgabe eines Werts des jeweiligen Typs auf der Konsole)

Beachten Sie bei der Umsetzung, dass Polymorphie in C++ etwas anders funktioniert als in Java.

Andere mit C++ verbundene Konzepte wie beispielsweise Präprozessor, Header-Files, Pointer, Templates, Sichtbarkeiten in Klassen, Trennung Deklaration/Implementierung bei Klassen (Trennung .h und .cpp) brauchen Sie nicht umsetzen.

Sie finden im Starter-Projekt einige Beispiele, die mindestens umgesetzt werden sollten und die Sie zum Testen Ihres Interpreters nutzen können. Beachten Sie, dass diese Sammlung nicht vollständig ist, und dass Sie selbst weitere Testbeispiele definieren sollen.

2 Projektvorstellung: Walk-Through statt Präsentation

Stellen Sie Ihr Projekt am Semesterende vor (Termine siehe Fahrplan und Ankündigung).

Jedes Team hat dafür **15 Minuten** Zeit.

Gehen Sie dabei am Code durch Ihr Projekt und diskutieren Sie relevante Teile, mindestens aber:

- Grammatik
- AST
- Semantische Analyse
- Interpreter

Demonstrieren Sie die Funktionsfähigkeit mit Ihren C++-Codebeispielen.

Sie sollen keine Folien erstellen. Die Präsentation soll live in der IDE erfolgen.

3 Abgabeformat

Geben Sie wie bei den anderen Blättern auch pro Person ein individuelles Post Mortem im ILIAS ab. Geben Sie dabei im Post Mortem unbedingt die URL zu Ihrem Team-Repo an. Denken Sie daran, dass dieses Repo öffentlich einsehbar sein muss.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 69c2d0e (homework: finalize B08 (#380), 2025-10-04)